```
import time
import os
import gzip
import urllib.request
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score


def load_fashion_mnist_direct():
    """Downloads raw Fashion-MNIST byte files directly with reliable
retries."""
    base_url =
"https://github.com/zalandoresearch/fashion-mnist/raw/master/data/
fashion/"
    files = {
        "X": "train-images-idx3-ubyte.gz",
        "y": "train-labels-idx1-ubyte.gz"
    }

    os.makedirs("./data_fashion", exist_ok=True)


    img_path = os.path.join("./data_fashion", files["X"])
    if not os.path.exists(img_path):
        print("Downloading Fashion-MNIST images directly...")
        urllib.request.urlretrieve(base_url + files["X"], img_path)


    lbl_path = os.path.join("./data_fashion", files["y"])
    if not os.path.exists(lbl_path):
        print("Downloading Fashion-MNIST labels directly...")
        urllib.request.urlretrieve(base_url + files["y"], lbl_path)


    with gzip.open(img_path, 'rb') as f:
        X = np.frombuffer(f.read(), dtype=np.uint8,
offset=16).reshape(-1, 784)


    with gzip.open(lbl_path, 'rb') as f:
        y = np.frombuffer(f.read(), dtype=np.uint8, offset=8)
```

```
    return X.astype("float32") / 255.0, y.astype(int)

print("Loading Fashion-MNIST dataset...")
try:
    X, y = load_fashion_mnist_direct()
    print("Dataset successfully loaded!")
except Exception as e:
    print(f"Direct load failed ({e}). Falling back to OpenML...")
    from sklearn.datasets import fetch_openml
    fashion = fetch_openml(name="Fashion-MNIST", version=1,
as_frame=False, parser="auto")
    X = fashion.data.astype("float32") / 255.0
    y = fashion.target.astype(int)


X, _, y, _ = train_test_split(X, y, train_size=10000, random_state=42,
stratify=y)

class_names = [
    "T-shirt", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"
]


plt.figure(figsize=(10, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X[i].reshape(28, 28), cmap="gray")
    plt.title(class_names[y[i]])
    plt.axis("off")
plt.suptitle("Fashion MNIST Sample Classes", fontsize=14)
plt.tight_layout()
plt.show()


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# ============================================
# 4. Define & Train SVM Pipelines
# ============================================
pipelines = {
    "Linear": Pipeline([
        ("scaler", StandardScaler()),
        ("svm", SVC(kernel="linear", C=1.0, cache_size=1000))
    ]),
    "Polynomial": Pipeline([
        ("scaler", StandardScaler()),
        ("svm", SVC(kernel="poly", degree=3, C=1.0, cache_size=1000))
```

```
    ]),
    "RBF": Pipeline([
        ("scaler", StandardScaler()),
        ("svm", SVC(kernel="rbf", C=1.0, gamma="scale",
cache_size=1000))
    ])
}

results = []

print("\nTraining models...")
for name, pipeline in pipelines.items():
    start_train = time.time()
    pipeline.fit(X_train, y_train)
    train_time = time.time() - start_train

    start_predict = time.time()
    predictions = pipeline.predict(X_test)
    predict_time = time.time() - start_predict

    results.append({
        "Kernel": name,
        "Accuracy": accuracy_score(y_test, predictions),
        "Precision": precision_score(y_test, predictions,
average="weighted", zero_division=0),
        "Recall": recall_score(y_test, predictions,
average="weighted", zero_division=0),
        "F1-Score": f1_score(y_test, predictions, average="weighted",
zero_division=0),
        "Train Time (sec)": round(train_time, 2),
        "Inference Time (sec)": round(predict_time, 2)
    })

# ========================================
# 5. Display Performance Summary Table
# ========================================
results_df = pd.DataFrame(results)
print("\n--- Kernel Performance Comparison ---")
print(results_df.round(4).to_string(index=False))

best_row = results_df.loc[results_df["F1-Score"].idxmax()]
print("\n" + "="*40)
print(f" Best Performing Kernel: {best_row['Kernel']}")
print(f" Accuracy: {best_row['Accuracy']:.4f}")
print(f" F1-Score: {best_row['F1-Score']:.4f}")
print("="*40)

# ========================================
# 6. Plot Metrics Bar Graph
# ========================================
```

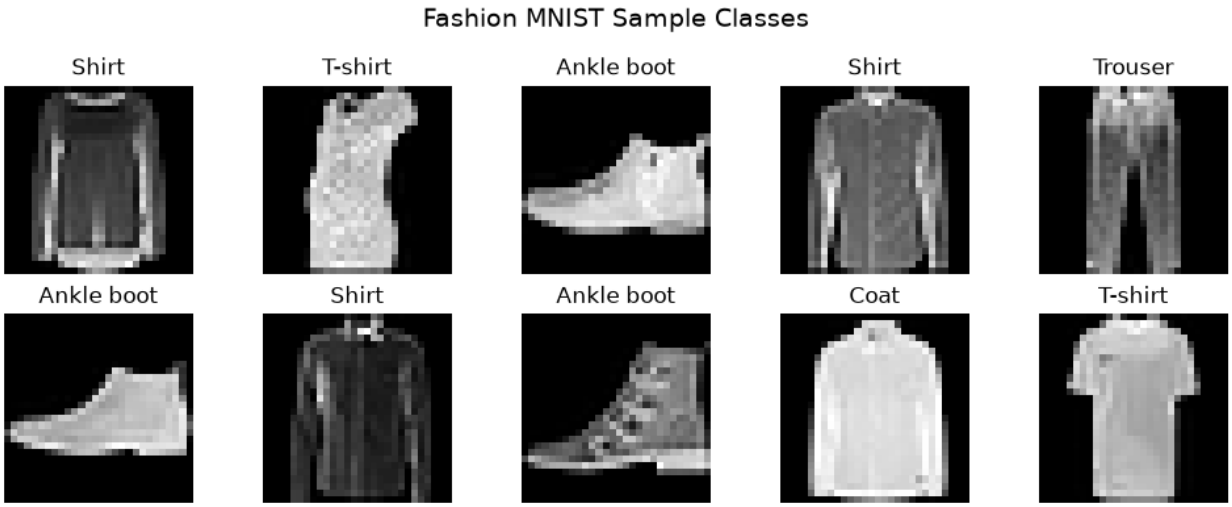
```
fig, ax = plt.subplots(figsize=(10, 6))
metrics_df = results_df.set_index("Kernel")[["Accuracy", "Precision",
"Recall", "F1-Score"]]
metrics_df.plot(kind="bar", ax=ax, ylim=(0, 1.05), rot=0)

plt.title("Fashion MNIST: SVM Kernel Performance Comparison",
fontsize=14, fontweight="bold")
plt.ylabel("Score", fontsize=12)
plt.xlabel("Kernel Type", fontsize=12)
plt.grid(axis="y", linestyle="--", alpha=0.6)
plt.legend(loc="lower right", frameon=True)

for p in ax.patches:
    height = p.get_height()
    if height > 0:
        ax.annotate(f"{height:.2f}",
                    (p.get_x() + p.get_width() / 2., height),
                    ha="center", va="bottom", fontsize=8, xytext=(0,
3),
                    textcoords="offset points")

plt.tight_layout()
plt.show()
```

```
Loading Fashion-MNIST dataset...
Dataset successfully loaded!
```




```
Training models...

--- Kernel Performance Comparison ---
    Kernel  Accuracy  Precision  Recall  F1-Score  Train Time (sec)
Inference Time (sec)
```

```
    Linear     0.7925      0.7949   0.7925     0.7933                26.04
7.80
Polynomial     0.8050      0.8290   0.8050     0.8123                40.38
10.22
       RBF     0.8410      0.8404   0.8410     0.8393                30.49
15.91

========================================
 Best Performing Kernel: RBF
 Accuracy: 0.8410
 F1-Score: 0.8393
========================================
```

